Machine Learning (CAI-262)

Objectives:

- **Soft SVM**
- **Linear and Nonlinear SVM**
- **Kernel trick**

Soft SVM:

There might be points that could be inside both margins. In this case, we could define two types of SVM, hard and soft margin SVM.

The hard margin SVM deals with the situation where the support vector is exactly outside both margins and there is no point between the margins. While soft margin SVM relaxes this condition and tolerates some points between both margins.

All the derivation in hard SVM were based on the requirement that the data be linearly separable but truth is that practical Problems are Non-Separable (linearly).

Through SVM Non-separable data can be handled by relaxing the constraint of hard SVM

$$y_i\left(\mathbf{w}^T\mathbf{x}+b\right)\geq 1$$

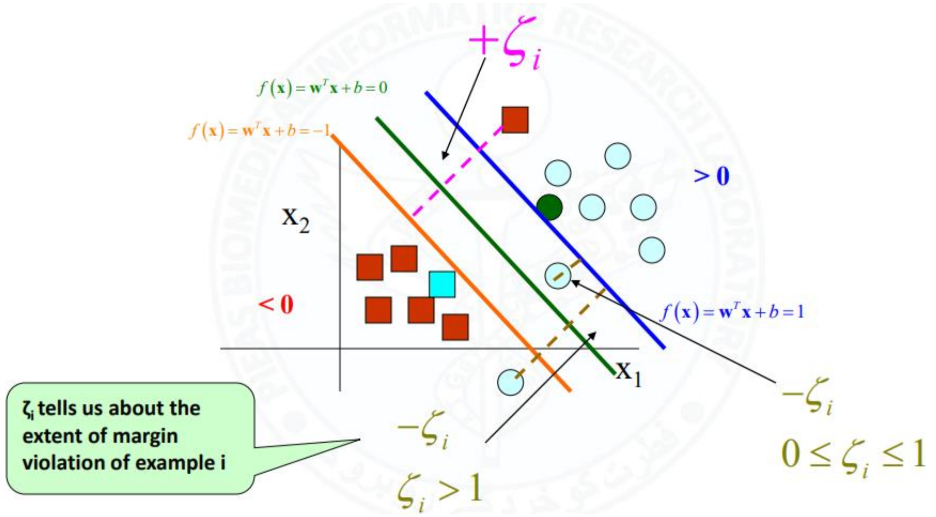
By introducing a slack variable

$$\left.\begin{aligned}&\mathbf{w}^T\mathbf{x}^{(i)}+b\geq 1-\zeta_i && \forall y_i=+1\\&\mathbf{w}^T\mathbf{x}^{(i)}+b\geq -1+\zeta_i && \forall y_i=-1\end{aligned}\right\}$$

$$\zeta_i\geq 0$$

Combinedly written as

$$y_i\left(\mathbf{w}^T\mathbf{x}^{(i)}+b\right)\geq 1-\zeta_i$$



So we can write overall optimization problem as follows

$$\min_{\mathbf{w},b} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{N}\zeta_i$$
$$s.t. \quad y_i\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \geq 1 - \zeta_i$$
$$\zeta_i \geq 0$$

Weight of the penalty due to margin violation

Or

$$\min_{\mathbf{w},b} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{N}\zeta_i$$
$$s.t. \quad 1 - \zeta_i - y_i\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \leq 0$$
$$-\zeta_i \leq 0$$

The objective function now optimizes two conflicting objectives: Maximization of the margin and minimization of the margin violations

C is the weight of the penalty of the term representing margin violation.

If C is small, then more margin violations will occur.

If C is large, lesser margin violations will occur.

C can be found out through cross-validation

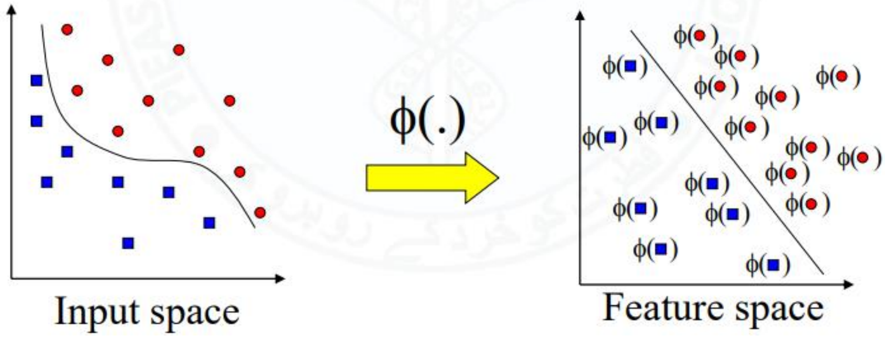
Types of SVM

SVM can be of two types:

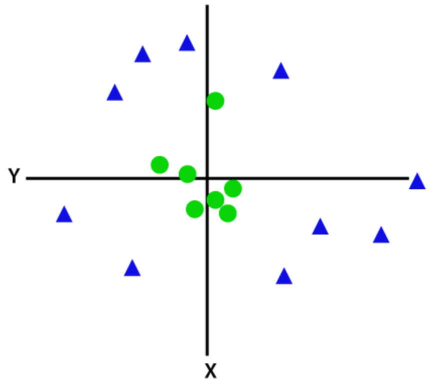
- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Nonlinear data separation through Transformation

Given a classification problem with a nonlinear boundary, we can, at times, find a mapping or transformation of the feature space which makes the classification problem linear separable in the transformed space.



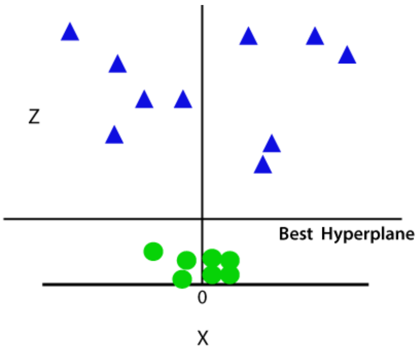
If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



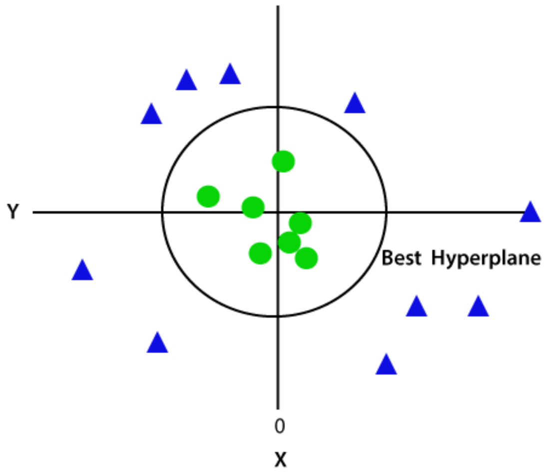
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as:

$z=x^2 +y^2$

By adding the third dimension, the sample space will become as below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with z=1, then it will become as:



Hence we get a circumference of radius 1 in case of non-linear data.

Examples of transformation

Example 1:

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} x_1 \\ x_2 \\ x_1 x_2 \end{bmatrix}$$

Example 2:

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} (x_1)^2 \\ (x_2)^2 \\ \sqrt{2 x_1 x_2} \end{bmatrix}$$

Example 3:

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} (x_1)^3 \\ (x_2)^3 \\ \sqrt{3}(x_1)^2 x_2 \\ \sqrt{3}(x_2)^2 x_1 \end{bmatrix}$$

Kernel Function:

Transformations can be used to make the data linearly separable. But it may not always be possible to find a transformation in this case we can use soft SVM.

The process of this feature transformation is called **Kernel Function.**

$$\max_{\alpha_i, \beta_i \geq 0} \left\{ \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \mathbf{x}^{(i)^T} \mathbf{x}^{(j)} \right\}$$

$$s.t \qquad 0 \leq \alpha_i \leq C, \qquad \sum_{i=1}^{N} \alpha_i y_i = 0$$

In above equation we can replace $x^{(i)^T} x^{(j)}$ with inner product of transformed features $\phi(x^{(i)})^T \phi(x^{(j)})$.

$$\max_{\alpha_i, \beta_i \geq 0} \left\{ \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \, k(i,j) \right\}$$

$$s.t \qquad 0 \leq \alpha_i \leq C, \qquad \sum_{i=1}^{N} \alpha_i y_i = 0$$

Since $\langle x^{(i)}, x^{(j)} \rangle$ is always a scalar , we can actually use a function called Kernel function $k(i,j)$ to map the two examples to a scalar value.

So kernel is a measure of how similar the two examples are whether they belong to the same class or not.

Some important kernel functions are given below.

- We can use arbitrary kernels, for example ...
  - The dot-product kernel
    - $K(\boldsymbol{a}, \boldsymbol{b}) = \boldsymbol{a}^T \boldsymbol{b}$
      - Feature transform: $\boldsymbol{\phi}(\boldsymbol{a}) = \boldsymbol{a}$
  - The homogenous polynomial kernels
    - $K(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a}^T \boldsymbol{b})^{\boldsymbol{p}}$, $\boldsymbol{p}$ is called degree
      - For $\boldsymbol{p} = \mathbf{2}$ and 2D data, $\boldsymbol{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$: $\boldsymbol{\phi}(\boldsymbol{a}) = \begin{bmatrix} a_1^2 \\ a_2^2 \\ \sqrt{2}a_1 a_2 \end{bmatrix}$
  - The Radial Basis Function (RBF) Kernel
    - $K(\boldsymbol{a}, \boldsymbol{b}) = e^{-\frac{\|\boldsymbol{a}-\boldsymbol{b}\|^2}{2\sigma^2}}$, $\boldsymbol{\sigma}$ controls the spread of the Gaussian
      - Feature transform?
- The RBF and Homogenous Polynomial kernels implement non-linear boundaries